ProductServlet.java

```
package com.harisudhan.harisudhanmart.controller;

import com.harisudhan.harisudhanmart.dto.ProductRequestDTO;
import com.harisudhan.harisudhanmart.exception.NotFoundException;
import com.harisudhan.harisudhanmart.exception.ValidationException;
import com.harisudhan.harisudhanmart.model.Product;
import com.harisudhan.harisudhanmart.service.ProductService;
import com.harisudhan.harisudhanmart.util.JsonUtil;
import java.io.IOException;
import java.sql.SQLException;
import java.util.List;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

/**
 * F2 (seller CRUD) and F3 (buyer browse/search). GET is public; create/update/delete require
 * a SELLER session (enforced here + AuthFilter covers the session-presence check).
 */
@WebServlet(urlPatterns = {
        "/api/v1/products", "/api/v1/products/create", "/api/v1/products/update", "/api/v1/products/delete"
})
public class ProductServlet extends BaseServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        ProductService service = new ProductService(daoFactory().productDAO());
        try {
            String id = req.getParameter("id");
            if (id != null) {
                writeJson(resp, HttpServletResponse.SC_OK, service.findByIdOrThrow(Long.parseLong(id)));
                return;
            }
            String sellerId = req.getParameter("sellerId");
            if (sellerId != null) {
                writeJson(resp, HttpServletResponse.SC_OK, service.findBySeller(Long.parseLong(sellerId)));
                return;
            }
            List<Product> results = service.search(req.getParameter("keyword"), req.getParameter("category"));
            writeJson(resp, HttpServletResponse.SC_OK, results);
        } catch (NotFoundException e) {
            writeError(resp, HttpServletResponse.SC_NOT_FOUND, "NOT_FOUND", e.getMessage());
        } catch (SQLException e) {
            writeError(resp, HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "DB_ERROR", "Lookup failed");
        }
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        HttpSession session = req.getSession(false);
        Long userId = currentUserId(session);
        String role = currentRole(session);
        if (!"SELLER".equals(role)) {
            writeError(resp, HttpServletResponse.SC_FORBIDDEN, "FORBIDDEN", "Seller role required");
            return;
        }

        ProductService service = new ProductService(daoFactory().productDAO());
        ProductRequestDTO dto = JsonUtil.gson().fromJson(req.getReader(), ProductRequestDTO.class);
        String path = req.getServletPath();
        try {
```

```
            if (path.endsWith("/create")) {
                writeJson(resp, HttpServletResponse.SC_CREATED, service.create(userId, dto));
            } else if (path.endsWith("/update")) {
                long id = Long.parseLong(req.getParameter("id"));
                writeJson(resp, HttpServletResponse.SC_OK, service.update(id, userId, dto));
            } else if (path.endsWith("/delete")) {
                long id = Long.parseLong(req.getParameter("id"));
                service.delete(id, userId);
                writeJson(resp, HttpServletResponse.SC_OK, null);
            } else {
                writeError(resp, HttpServletResponse.SC_NOT_FOUND, "NOT_FOUND", "Unknown product endpoint");
            }
        } catch (ValidationException e) {
            writeError(resp, HttpServletResponse.SC_BAD_REQUEST, e.getFieldErrorCode(), e.getMessage());
        } catch (NotFoundException e) {
            writeError(resp, HttpServletResponse.SC_NOT_FOUND, "NOT_FOUND", e.getMessage());
        } catch (SQLException e) {
            writeError(resp, HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "DB_ERROR", "Operation failed");
        }
    }
}
```